Semantic Geometry and Corrective Navigation: Event Semantics, Spatial Projection, and Admissible Continuation

Flyxion
Independent Researcher

October 2026

Abstract

This document specifies a navigable information environment in which spatial position expresses relationships but never establishes them. Authority is placed in an append-only event ledger. A semantic graph is derived from the currently effective events, and a spatial projection is derived from that graph by a reproducible layout procedure. Relevance, evidential support, and admissibility of continuation are kept as three distinct relations that are never merged into a single score. Corrections are first-class events that record the scope and completeness of the dependency traversal that accompanied them. The specification is stated as a set of independently testable invariants, so that any later experimental program has a fixed definition of what the system is meant to preserve.

1 Scope and status

This is a conceptual specification and a proposed design direction. It does not describe an implemented system. Several earlier formulations motivate it: a principle of routing by meaning rather than by address, a coherence-first alternative to social media built on persistent semantic objects, a distinction between routing and adjudication, and a treatment of correction as a geometric property of an information structure. The dates, titles, and exact wording attributed to those earlier works have not been verified against the original manuscripts, and nothing in this specification depends on them. Any ledger entry that cites them must carry the status `unverified` until the citation is checked. Historical verification is a separate task and is deliberately excluded from this document.

2 Central requirement

The specification is organized around one requirement.

> Every authoritative relationship must be typed, attributable, and reconstructible from an event history. Spatial proximity may express such relationships, but must never establish them. Retrieval proposes candidates, adjudication determines evidential status, and admissibility governs possible continuations.

The three sentences of the requirement assign three different jobs to three different mechanisms. Retrieval, whether by embedding similarity, keyword match, or any other method, can only nominate a candidate relation. Adjudication, carried out through explicit support and defeat events, is the only route by which a claim acquires an evidential status. Admissibility is a constraint on what may come next, and it is computed from the current state of evidence rather than asserted by the interface. None of the three is permitted to stand in for another.

3 Architectural layers

The system has three layers whose authority is strictly ordered, together with an external version-control substrate.

Layer	Contents	Authority	Mutability
Event ledger $\mathcal{L}$	Typed, attributed events: assertions, relations, support, defeat, repair, layout constraints, admissibility conditions	Authoritative for semantic provenance	Append only
Semantic graph $\mathcal{G}_v$	Interpretation of the effective events at ledger version v	None of its own; derived	Recomputed from $\mathcal{L}$
Spatial projection P	Positions and visual encodings of graph elements	None of its own; derived	Recomputed or cached
Git	Content-addressed history of Git-managed artifacts	Authoritative for those artifacts' version history	Append only

The ledger records what was asserted and how it was later changed. The graph is a derived interpretation of the currently effective events. The projection is a navigational representation of the graph. Neither the graph nor the projection may independently establish a historical fact. If the two derived layers were deleted, they could be rebuilt from the ledger, and the rebuilt graph would be identical to the deleted one.

4 The event ledger

4.1 Schema

Each event carries the following fields.

```
event {
  id            // content-addressed identifier
  t             // timestamp
  actor         // human, agent, or named process (including
                // any retriever, with model identifier and version)
  kind          // assert | relate | unrelate | support | defeat
                // | repair | constrain | admit
  subject, object
  rel_type      // shares_concept | depends_on | shares_obligation
                // | admits_transform | cites
```

```
  channel       // relevance | support | admissibility
  basis         // list of event ids or evidence references;
                // required for every kind except assert
  caused_by     // event id, or null
  supersedes    // event id, or null
  git_ref       // commit hash, or null
  status        // provisional | established | unverified
}
```

4.2 Event kinds

Kind	Meaning
`assert`	Introduces an object or claim, with an actor but no required basis.
`relate`	Records a typed relation between two objects in exactly one channel.
`unrelate`	Withdraws an earlier relation by superseding it.
`support`	Records evidence for a claim, with the evidence cited in the basis.
`defeat`	Records evidence against a claim or against a support event. It alters the claim's epistemic status. It does not delete the relation.
`repair`	Records the correction of a defective event, relation, or object (Section 8).
`constrain`	Records a user-requested layout constraint such as pinning, grouping, or separating (Section 6).
`admit`	Declares a continuation admissible, conditional on stated evidence requirements.

Computed positional change is deliberately absent from this list. A change in position is a derived result of the projection procedure and is never itself an event.

4.3 Status values

An event with status `provisional` has been proposed but not adjudicated. Events nominated by retrieval processes enter as provisional. An event with status `established` has met whatever adjudication policy the deployment declares. An event with status `unverified` cites material, such as an earlier manuscript, whose existence or content has not been checked. Provisional and unverified events may be displayed, but they must be rendered distinguishably from established ones.

5 The semantic graph

Let $\mathcal{L}_v$ denote the ledger at version v. An event is *effective* at v if no later event in $\mathcal{L}_v$ supersedes it, either through its `supersedes` field or through an `unrelate` targeting it. The graph is the interpretation of the effective set:

$$\mathcal{G}_v = \mathrm{Interp}\big(\mathrm{Eff}(\mathcal{L}_v)\big).$$

A `defeat` event does not remove the defeated relation from $\mathrm{Eff}(\mathcal{L}_v)$. It changes the support status computed for the relevant claim. This keeps a defeated claim visible together with the record of its defeat, which is what makes later inspection of the correction possible.

6 Spatial projection

6.1 Constraints and computed placement

The projection distinguishes two sources of position. A *layout constraint* is an event: a user or agent asks that an object be pinned, grouped with others, or kept apart from others, and the request is attributed and recorded. A *computed placement* is the output of a layout procedure applied to the graph and the active constraints. Writing C_v for the effective constraint events, the projection is

$$P = \mathrm{Layout}\big(\mathcal{G}_v,\ C_v;\ a,\ \theta,\ s\big),$$

where a identifies the layout algorithm and its version, θ its parameters, and s its random seed. The tuple (v, a, θ, s) is the *projection key*.

6.2 Reproducibility

A projection is reproducible if, given its key, the same positions are obtained. Cached positions are permitted as an optimization, but a cache is keyed by the projection key and discarded when any component changes. A cached position is never authoritative, and it is never cited as the reason a relation holds.

When two projections of the same object differ, the system must be able to attribute the difference to one or more of three sources: a change in the effective graph, a change in the effective constraints, or a change in the algorithm, its parameters, or its seed. This attribution is the operational form of the requirement that every movement be explicable. A movement supported by a `relate` event but dependent on an unrecorded layout decision would fail the requirement, and the projection key exists to prevent that case.

6.3 Retrieval and embeddings

Embedding similarity may nominate a candidate relation by emitting a provisional `relate` event whose actor is the retriever, with the model identifier and version recorded in the basis. Such an event may influence layout only according to a declared policy, and by default a provisional relation is rendered as provisional and does not license a position on its own.

7 Three relations

7.1 Definitions

Relevance, support, and admissibility are distinct relations that may hold over the same objects.

- Relevant(x, u) holds when object x bears on the context u, which may be a user, a task, or a current activity. Relevance is relative to a context and is computed from the graph and that context.
- Supported(x) is the evidential status of claim x, computed from the effective `support` and `defeat` events. It takes values in a small discrete set such as unassessed, supported, contested, and defeated, and it is not a scalar.
- Admissible$(c \mid \mathcal{G}_v)$ holds when a continuation c is permitted at version v, meaning that every evidence requirement in the governing `admit` events is met by effective support events.

7.2 Non-implication

The specification adopts the following non-implications as requirements on any component that consumes these relations:

$$\begin{aligned} \text{Relevant}(x, u) &\not\Rightarrow \text{Supported}(x), \\ \text{Supported}(x) &\not\Rightarrow \text{Relevant}(x, u), \\ \text{Admissible}(c) &\not\Rightarrow \text{Supported}(c). \end{aligned}$$

A continuation may be admissible because its stated requirements are met even though the claim it produces is not yet supported, and a well-supported claim may be inadmissible as a next step in the current context. A system that collapses these relations becomes a recommendation engine that mistakes salience for truth.

7.3 No scalar collapse

An interface may compute a ranking for presentation, but a ranking is a presentation function. It must declare its inputs, and it must not be labelled with epistemic terms such as confidence, credibility, or truth. No ranking value is written to the ledger as evidence.

8 Correction

8.1 Repair events

A `repair` event targets an earlier event, relation, or object and records the following in addition to the common fields: the nature of the defect, an optional replacement, the *traversal scope*, the *graph version* on which the traversal ran, the *completeness status*, and the set of dependents found. Dependency is traversed over a declared subset D of relation types, by default `depends_on` and `shares_obligation`.

8.2 Completeness

A traversal can identify known dependents but cannot in general establish that every affected object has been found. The completeness status therefore takes one of three values.

- `complete`: every D-edge reachable within the declared scope was traversed, and the scope is declared closed under D.
- `partial`: the scope was limited by depth, access rights, or the absence of reachable federated peers.
- `unknown`: reachability could not be assessed.

In a federated deployment, a repair whose scope extends beyond the locally accessible graph is `partial` unless each remote peer has confirmed its own traversal against a recorded graph version. A completeness status is itself a claim, and it is revisable. When a dependent is discovered later, the original repair record is not rewritten. A follow-up `repair` event references it and extends the dependent set.

8.3 Consequences are events

For each dependent found, the repair produces a new status event. The visible change in the interface, such as a re-sealed or collapsed passage, is therefore the rendering of a logged consequence and not a styling decision.

9 Relation to Git

Git remains the authoritative record for the version history of Git-managed artifacts, including what changed, who changed it, and in which branch. The ledger does not duplicate this. Not every semantic event corresponds to a source change, so `git_ref` is nullable. The ledger is explicitly responsible for semantic provenance, meaning the assertions, relations, evidence, and corrections about artifacts and claims, including those with no commit behind them. Where an event concerns a Git-managed artifact, `git_ref` anchors it to the relevant commit, and the two records are consulted together.

10 Interface obligations

The interface is a projection of the graph and the ledger, and it inherits the following obligations.

1. **Explanation.** For any displayed relation, and for any change in position between two projections, the system returns the chain of events and the attribution described in Section 6.
2. **Channel encodings.** Relevance, support status, and admissibility use separate visual encodings. A blended similarity encoding is not permitted.
3. **Typed passages.** Each relation type has its own architectural form. Passages are physically traversable in both directions and semantically directed from the citing object to the cited object, with one-way passages reserved for irreversible relations.
4. **Doors.** A possible continuation appears as a door that remains dimmed until its evidence requirements are met. Door state is computed from the `admit` events and current support, and it is never stored as a position.
5. **Path history.** The rear view shows the actual path taken by the user. It is a navigation record distinct from the semantic ledger and from citation ancestry.
6. **Text projection.** A text-mode navigator is another projection of the same graph and ledger and never maintains a second history.

11 Provisional diagnostic: correction coupling

An earlier conceptual formulation treats the sensitivity of a system's internal evaluation E to an external corrective channel O as a coupling $\rho = \partial E/\partial O$, with $\rho \to 0$ marking a system whose evaluations no longer respond to external correction. This remains a research hypothesis and not an invariant of the specification. A literal derivative requires an operational definition of both variables, a perturbation procedure, and assumptions that distinguish a change in the corrective channel from an unrelated change in the evaluation. Until those are supplied, the

diagnostic may be explored as an experiment but must not be used to gate any behavior of the system.

12 Invariants and test obligations

Each invariant is stated so that it can be tested independently.

Invariant 1 (Ledger primacy)**.** Every authoritative relation is the image of one or more effective ledger events. The graph and the projection hold no fact that cannot be derived from the ledger.

Test: delete the derived layers, rebuild from the ledger, and compare.

Invariant 2 (Proximity is not authority)**.** No displayed adjacency establishes a relation. Every displayed relation resolves to its event identifiers.

Test: for a sample of displayed relations, resolve each to events, and confirm that adjacency with no supporting event is never rendered as a relation.

Invariant 3 (Channel separation)**.** Each relation belongs to exactly one channel. Promotion from one channel to another occurs only through an explicit event with a basis.

Test: audit the ledger for relations lacking a channel, and for state changes across channels that lack a promoting event.

Invariant 4 (Reproducible projection)**.** Computed positions are reproducible from the projection key (v, a, θ, s). Layout requests are events, and computed positions are derived and may be cached without becoming authoritative.

Test: regenerate a projection from its key and compare, and confirm that every positional difference between versions is attributed to graph, constraint, or algorithm changes.

Invariant 5 (Retrieval nominates)**.** Retrieval output enters only as provisional events with the retriever recorded as actor. It does not independently license a relation, a position, or a status.

Test: disable the retriever and confirm that no established relation changes.

Invariant 6 (Repair accountability)**.** Every repair records its target, scope, graph version, completeness status, and dependents found. Each dependent receives a logged status event.

Test: inject a known defective assertion with known dependents, apply a repair, and compare the recorded dependents with the known set.

Invariant 7 (Honest completeness)**.** A repair is recorded as `complete` only when the declared scope is closed under D and fully traversed. Scopes extending into unconfirmed federated peers are `partial`.

Test: plant a dependent outside the declared scope and confirm that the repair is not recorded as complete.

Invariant 8 (Provenance split)**.** `git_ref` is nullable. Git is authoritative for the history of Git-managed artifacts, and the ledger is authoritative for semantic provenance.

Test: confirm that semantic events without commits are representable, and that no Git history is copied into the ledger.

Invariant 9 (Derived admissibility)**.** Door state is computed from `admit` events and current support, and is not stored.

Test: add a supporting event and confirm that the door state changes with no edit to any stored position.

Invariant 10 (Explanation on demand)**.** For any displayed relation or position change, the system returns the supporting event chain.

Test: sample displayed relations and movements, and score each for the existence and correctness of the returned chain.

Invariant 11 (Unverified marking)**.** Any event citing unchecked external material carries the status `unverified`, and the interface renders it distinguishably.

Test: confirm that no unverified event is rendered with the same encoding as an established one.

13 Forward reference: experimental program

With these obligations fixed, an experimental program can compare five conditions: directory browsing, keyword search, embedding-only navigation, typed graph navigation, and the full corrective geometry. The measures of interest extend beyond retrieval performance to whether users can explain relationships, discover dependencies, recover from incorrect assertions, and identify the consequences of repairs. Because each invariant above carries its own test, a failure in a condition can be traced to a specific violated obligation and not merely to a poorer outcome score.